

Frontend стек для Roblox-подобной web-платформы

Общая архитектура

Проект делится на 2 основные части:

1. Web App (UI)
2. Game Runtime (3D игра)

1. Web App

Обычная frontend часть:

- логин
- регистрация
- список серверов
- меню
- инвентарь
- чат
- настройки
- магазин
- social features

2. Game Runtime

Игровая часть:

- 3D мир
 - rendering
 - physics
 - multiplayer
 - entities
 - asset streaming
 - simulation
-

Основной стек

React

Что это

Библиотека для построения UI.

Для чего используется

React отвечает за:

- интерфейсы
 - окна
 - HUD
 - меню
 - inventory
 - social UI
 - editor UI
 - launcher
-

Что НЕ делает

React не должен:

- управлять игровым миром
 - обновлять entities
 - хранить realtime game state
 - заниматься rendering loop
-

Как использовать

React используется как оболочка вокруг игрового runtime.

Пример структуры:

```
React
├─ Login
├─ Main Menu
├─ Chat
├─ Inventory
└─ Game Canvas
```

TypeScript

Что это

JavaScript с типами.

Почему обязателен

Без TypeScript большой frontend проект становится сложно поддерживать.

TypeScript:

- даёт типизацию
 - улучшает IDE
 - уменьшает runtime ошибки
 - делает код более поддерживаемым
-

Как использовать

Весь frontend проект писать на TypeScript:

- React
 - Babylon
 - networking
 - runtime systems
-

Vite

Что это

Система сборки frontend проекта.

Для чего нужен

Vite:

- запускает dev server
 - собирает проект
 - делает hot reload
 - компилирует TypeScript
-

Почему Vite

Потому что:

- очень быстрый
 - проще Webpack
 - отлично работает с React и TS
 - удобен для game/web проектов
-

Babylon.js

Что это

Полноценный 3D движок для браузера.

Что делает

Babylon отвечает за:

- rendering
 - scene management
 - lighting
 - shadows
 - cameras
 - materials
 - animation
 - model loading
 - post-processing
 - particles
-

Почему подходит

Babylon хорошо подходит для:

- web игр
 - sandbox платформ
 - browser multiplayer
 - dynamic asset loading
 - streaming worlds
-

Как использовать

Babylon запускается внутри canvas:

```
React UI
  ↓
Babylon Canvas
  ↓
Game Runtime
```

WebGPU

Что это

Современный graphics API браузера.

Для чего нужен

Через WebGPU Babylon рендерит 3D.

Почему WebGPU

По сравнению с WebGL:

- лучше производительность
 - современный pipeline
 - compute shaders
 - ближе к Vulkan/DX12
-

Как использовать

Babylon умеет работать поверх WebGPU автоматически.

Обычно:

```
Babylon
  ↓
WebGPU
  ↓
GPU
```

Rapier

Что это

Physics engine.

Для чего нужен

Rapier считает:

- столкновения
 - rigid bodies
 - гравитацию
 - physics simulation
-

Почему Rapier

Потому что:

- быстрый
 - работает через WASM
 - хорошо подходит для web
 - современная архитектура
-

Как использовать

Babylon может интегрироваться с Rapier.

Tailwind CSS

Что это

CSS framework.

Для чего нужен

Используется для быстрого создания UI:

- меню
- окна
- панели
- overlays
- editor UI

Почему Tailwind

Потому что:

- очень быстрый для разработки
- удобно масштабировать UI
- хорошо работает с React
- меньше проблем с CSS архитектурой

Zustand

Что это

Библиотека для хранения frontend состояния.

Для чего нужен

Используется для UI state:

- открыт ли inventory
- выбранный сервер
- настройки
- user session
- UI overlays

Что НЕ хранить

Не хранить:

- entities
- positions
- realtime world state
- physics state

Игровой мир должен жить отдельно от React.

IndexedDB

Что это

База данных браузера.

Для чего нужна

Используется как локальный cache:

- модели
 - текстуры
 - sounds
 - mode assets
 - downloaded content
-

Почему это важно

Без локального cache:

- всё будет скачиваться заново
 - долгие загрузки
 - высокий трафик
-

Web Workers

Что это

Потоки браузера.

Для чего нужны

Позволяют выносить тяжёлые задачи из main thread.

Что можно выносить

Например:

- physics
 - pathfinding
 - decompression
 - asset processing
 - navmesh generation
-

Общая схема frontend части

```
React
└─ UI
```


- └─ Menus
- └─ Chat
- └─ Inventory
- └─ Social

Babylon

- └─ Rendering
- └─ Scene
- └─ Animation
- └─ Materials
- └─ Asset Loading

Runtime

- └─ ECS
- └─ Simulation
- └─ Networking
- └─ Replication

Physics

- └─ Rapier

Что изучать в первую очередь

1. TypeScript

Основа всего frontend.

2. React

Для UI и платформенной части.

3. Babylon.js

Для 3D runtime.

4. WebGPU

На базовом уровне.

5. Browser APIs

- canvas
- workers
- IndexedDB
- audio
- networking

Что можно оставить на потом

- shaders
 - low-level rendering
 - custom rendering pipeline
 - advanced WebGPU internals
 - WASM optimizations
-

Главная мысль

Основная сложность Roblox-подобной платформы — не renderer.

Главные проблемы:

- multiplayer
- replication
- streaming
- sandboxing
- runtime architecture
- asset delivery
- scalability

3D renderer — только часть системы.